Часть 1 из 5: Основы классов и управления ресурсами (Билеты 1-4)

Билет 1. Описание класса в C++. Поля и методы. Внутренние и внешние методы. Разграничение доступа. Const.

1. Что такое класс?

Класс — это пользовательский тип данных (User-defined type). Это чертеж (blueprint), описывающий сущность:

- **Данные (Поля):** Из чего объект состоит.
- **Поведение (Методы):** Что объект умеет делать.

В C++ класс реализует принцип **инкапсуляции** — объединение данных и методов работы с ними, а также сокрытие внутренней реализации.

2. Поля и Методы

- **Поля (Data Members):** Переменные, объявленные внутри класса.
  - *Совет:* Используй префикс `m_` (например, `m_age`), чтобы отличать поля от локальных переменных.
- **Методы (Member Functions):** Функции, объявленные внутри класса. Они имеют доступ ко всем полям (даже `private`).
  - У каждого нестатического метода есть скрытый параметр `this` — указатель на текущий объект.

3. Внутренняя и внешняя реализация

- **Внутренняя (Inline):** Код пишется прямо в теле класса.
  - *Смысл:* Компилятор пытается встроить код в место вызова (inline).
  - *Где:* Геттеры, сеттеры, однострочные функции.
- **Внешняя:** В классе только прототип, код в `.cpp` файле (через `ClassName::method()`).
  - *Смысл:* Ускоряет компиляцию, уменьшает размер заголовков.

4. Спецификаторы доступа

1. **`private` (Закрытый):** Доступ только у методов класса и друзей. (По умолчанию в `class`).
2. **`public` (Открытый):** Доступ у всех. Это внешний интерфейс.
3. **`protected` (Защищенный):** Доступ у методов класса и **наследников**.

5. Константные методы

Если метод не меняет состояние объекта, он **обязан** быть `const`.

- **Синтаксис:** `void func() const { ... }`
- **Механика:** Внутри метода `this` меняет тип с `Type*` на `const Type*`.
- **Важность:** Только константные методы можно вызывать у константных объектов (`const Student s`).

Пример:

```
class BankAccount {
private:
    double m_balance; // Приватное поле
public:
    // Конструктор
    BankAccount(double b) : m_balance(b) {}

    // const метод (геттер)
    double getBalance() const {
        // m_balance = 0; // Ошибка компиляции!
        return m_balance;
    }

    // Обычный метод (сеттер)
    void deposit(double amount) {
        m_balance += amount;
    }
};
```

Билет 2. Конструкторы. Списки инициализаторов. Копирование.

1. Конструктор

Метод, вызываемый **автоматически** при создании объекта.

- Имя совпадает с именем класса.
- Не имеет возвращаемого типа (даже `void`).
- **Цель:** Перевести объект из «сырого» состояния (мусор в памяти) в валидное (инвариант соблюден).

2. Список инициализации (`: field(val)`)

Единственный правильный способ инициализации. Пишется после двоеточия.

- **Почему важно:**
    1. **Производительность:** Инициализирует поля сразу. Иначе сначала вызовется дефолтный конструктор поля, а потом оператор присваивания в теле (двойная работа).
    2. **Обязательно для:** Ссылок (`Type&`), констант (`const Type`) и объектов без дефолтного конструктора.
- *Нюанс:* Поля инициализируются строго в порядке их объявления в классе!

3. Конструктор по умолчанию

Конструктор без аргументов.

- Если вы не написали **никаких** конструкторов, компилятор создаст его сам (`Student() {}`).
- Если написали хоть один с параметрами — автогенерация отключается. Вернуть можно через `= default`.

4. Конструктор копирования

Создает копию объекта. Сигнатура: `ClassName(const ClassName& other)`.

- **Shallow Copy (Поверхностная):** Копирует биты (адреса указателей). Опасно, если есть динамическая память (приводит к Double Free).
- **Deep Copy (Глубокая):** Выделяет новую память и копирует содержимое.

Пример (Deep Copy):

```
class Array {
    int* data;
    int size;
public:
    Array(int s) : size(s) { data = new int[s]; } // Обычный ctor

    // Copy ctor
    Array(const Array& other) : size(other.size) {
        data = new int[size]; // Новая память
        std::copy(other.data, other.data + size, data); // Копия данных
    }
    ~Array() { delete[] data; }
};
```

Билет 3. Деструкторы. Необходимость собственных деструкторов.

1. Что такое деструктор?

Метод `~ClassName()`. Вызывается автоматически при уничтожении объекта.

- **Когда:** Выход из области видимости (стек), вызов `delete` (куча), завершение программы (статика).
- **Аргументов:** Нет.
- **Перегрузка:** Невозможна.

2. Зачем писать свой?

Если класс реализует концепцию **RAII** (Resource Acquisition Is Initialization) и владеет ресурсом:

1. Динамическая память (`new`).
2. Файловый дескриптор (`fopen`).
3. Сетевой сокет, мьютекс.

Если не написать деструктор с `delete/fclose`, произойдет **утечка ресурсов**.

3. Важные правила

1. **Порядок:** Обратен конструкторам (сначала разрушается наследник, потом база; сначала поля, потом сам класс).
2. **Исключения:** Деструктор **обязан** быть `noexcept`. Если он бросит исключение во время раскрутки стека (когда уже летит другое исключение), программа аварийно завершится (`std::terminate`).

Пример:

```
class FileWrapper {
    FILE* f;
public:
    FileWrapper(const char* name) { f = fopen(name, "r"); }
    ~FileWrapper() {
        if(f) fclose(f); // Обязательная очистка
    }
};
```

Билет 4. Дружественные функции и классы.

1. Суть (`friend`)

Механизм, позволяющий внешней функции или другому классу получить доступ к **`private`** и **`protected`** членам вашего класса.

- Это "ключи от квартиры" для избранных.

2. Зачем нужно?

1. **Перегрузка операторов:** Особенно `operator<<` (вывод в `cout`), так как левый операнд — это поток, а не наш класс.
2. **Тесная связь:** Паттерны (например, `Iterator` должен видеть кишки контейнера).
3. **Тесты:** Чтобы Unit-тесты могли проверять внутреннее состояние.

3. Три правила "НЕ"

1. Дружба **не** симметрична (Я твой друг ≠ Ты мой друг).
2. Дружба **не** транзитивна (Друг моего друга ≠ Мой друг).
3. Дружба **не** наследуется.

Пример:

```
class Box {
    int secret;
public:
    Box(int s) : secret(s) {}
    // Объявляем внешнюю функцию другом
    friend void printSecret(const Box& b);
};

// Функция не является методом, но видит private
void printSecret(const Box& b) {
    std::cout << b.secret << std::endl;
}
```

Вот **Часть 2** (Билеты 5–8). Здесь мы переходим от основ к более сложным механизмам: исключениям, перегрузке, шаблонам и полиморфизму.

Часть 2 из 5: Продвинутые механизмы C++ (Билеты 5-8)

Билет 5. Выброс исключений. Конструкторы. std::exception.

1. Механизм исключений

В C++ ошибки обрабатываются через прерывание потока выполнения.

- **`throw`:** Бросает объект-исключение.
- **Раскрутка стека (Stack Unwinding):** Автоматический процесс уничтожения всех локальных объектов (вызов их деструкторов) в обратном порядке создания, пока управление не дойдет до блока `catch`.

2. Исключения в конструкторах

Это **единственный** способ сообщить об ошибке при создании объекта (у конструктора нет `return`).

- **Главное правило:** Если конструктор бросил исключение, объект считается **не созданным**.
- **Последствие:** Деструктор *этого* объекта вызван не будет!
- **Опасность:** Если вы выделили память (`new`) внутри конструктора, а потом бросили исключение, память утечет (так как деструктор не сработает).
  - *Решение:* Используйте умные указатели (`std::unique_ptr`) внутри класса. Они очистятся автоматически.

3. Исключения в методах

Методы могут гарантировать безопасность:

- **Basic Guarantee:** Утечек нет, объект в валидном состоянии.
- **Strong Guarantee:** Либо успех, либо состояние не изменилось (транзакционность).
- **Noexcept:** Метод гарантирует отсутствие исключений.

4. Класс `std::exception`

Бросать нужно объекты, наследуемые от `std::exception` (библиотека `<stdexcept>`).

- `std::logic_error`: Ошибки в логике программы (индексы, аргументы).
- `std::runtime_error`: Ошибки среды (файлы, сеть).
- Ловить нужно **по ссылке** (`catch (const std::exception& e)`), чтобы избежать "срезки" (копирования только базовой части объекта).

Билет 6. Перегрузка операторов.

1. Суть

Придание стандартным операторам (`+`, `-`, `<<`) нового смысла для пользовательских типов. Делает код читаемым: `matrixC = matrixA + matrixB`.

2. Ограничения (Что нельзя)

1. Создавать новые операторы (например, `**`).
2. Менять приоритет и количество аргументов.
3. Перегружать операторы для встроенных типов (`int + int`).

3. Два способа реализации

- **А. Метод класса:** Левый операнд — всегда `*this`.
  - Обязательно для: `=`, `[]`, `()`, `->`.
- **Б. Внешняя функция (часто `friend`):**
  - Нужна, если левый операнд — *не наш класс* (например, `std::cout << obj` или `5 * vector`).
  - Нужна для симметрии (`obj + 5` и `5 + obj`).

4. Пример (`Vector2D`)

```
class Vector2D {
    int x, y;
public:
    Vector2D(int a, int b) : x(a), y(b) {}

    // 1. Метод класса: Оператор присваивания
    // Возвращаем ссылку на себя для цепочки a = b = c
    Vector2D& operator=(const Vector2D& other) {
        if (this == &other) return *this; // Защита от самоприсваивания
```

```
        x = other.x; y = other.y;
        return *this;
    }

    // 2. Внешняя функция (friend): Сложение
    friend Vector2D operator+(const Vector2D& lhs, const Vector2D& rhs);
};

// Реализация +
Vector2D operator+(const Vector2D& lhs, const Vector2D& rhs) {
    return Vector2D(lhs.x + rhs.x, lhs.y + rhs.y);
}
```

Билет 7. Шаблоны классов. Специализация.

1. Шаблоны (Templates)

Это механизм обобщенного программирования. Мы пишем "чертеж" класса, где тип данных заменен на параметр `T`.

- **Инстанцирование:** Компилятор генерирует реальный код класса только в момент использования (`vector<int>`, `vector<double>`).
- **Важно:** Код шаблонов (реализация) должен находиться в **заголовочных файлах (.h)**, иначе будет ошибка линковки.

2. Специализация

Иногда общий алгоритм не подходит для конкретного типа (например, `bool` или `char*`).

- **Полная специализация:** Переписываем класс целиком для одного типа. `template<> class Box<bool> { ... };`
- **Частичная специализация:** Уточняем поведение для группы типов (например, для всех указателей `T*`).

3. Вариативные шаблоны (Variadic Templates)

Появились в C++11. Позволяют принимать произвольное количество параметров типов. Используют эллипсис `...`.

- Пример: `std::tuple`, `printf`-подобные функции.
- Обрабатываются через рекурсию или свертку (Fold Expressions в C++17).

Пример:

```
template <typename T>
class Box { T val; }; // Общий

// Полная специализация для char*
template <>
class Box<char*> {
    char* val;
    // Тут нужна логика с выделением памяти и strcpy
};
```

Билет 8. Полиморфизм. Статический и Динамический.

1. Определение

Полиморфизм — способность функции или объекта обрабатывать данные разных типов. "Один интерфейс — много реализаций".

2. Статический полиморфизм (Compile-time)

Решение о том, какую функцию вызвать, принимается **при компиляции**.

- **Инструменты:** Перегрузка функций, Шаблоны.
- **Плюсы:** Быстро (нет накладных расходов в рантайме).
- **Минусы:** Увеличивает размер бинарного файла (Code Bloat), требует перекомпиляции при изменении.

3. Динамический полиморфизм (Run-time)

Решение принимается **во время выполнения**. Это "настоящий" ООП-полиморфизм.

- **Инструменты:** Наследование + Виртуальные функции (`virtual`) + Указатели/Ссылки на базовый класс.
- **Механизм:** `vtable` (таблица виртуальных методов).
- **Суть:** Мы работаем с `Animal*`, но если там лежит `Dog`, вызовется `Dog::bark()`.

4. Сравнение

Характеристика	Статический	Динамический
Когда решается	Компиляция	Выполнение
Скорость	Максимальная	Чуть медленнее (косвенный вызов)
Гибкость	Нужен исходный код	Можно подгружать плагины (DLL)
Ключевые слова	`template`, перегрузка	`virtual`, `override`, `dynamic_cast`

Конец Части 2. *Напиши "дальше", чтобы я отправил Часть 3 (Билеты 9-12).*

Вот **Часть 3** (Билеты 9–12). Здесь мы разбираем современный C++ (перемещение) и все нюансы наследования.

Часть 3 из 5: Семантика перемещения и Наследование (Билеты 9-12)

Билет 9. lvalue и rvalue. Конструктор перемещения.

1. Категории значений

- **lvalue (locator value):** Объект, у которого есть имя и адрес в памяти. Живет дольше одной строки. (Примеры: переменные `int x`, разыменованные указатели).
- **rvalue (right value):** Временный объект, который скоро умрет. Обычно не имеет имени. (Примеры: `5`, `x + y`, результат функции по значению).

2. Семантика перемещения (Move Semantics)

Введена в C++11. Позволяет не копировать тяжелые данные (Deep Copy), а "красть" их у временных объектов.

- **Идея:** Если объект всё равно сейчас удалится (rvalue), зачем копировать его данные? Давайте просто заберем у него указатель на буфер.

3. Реализация

1. **Конструктор перемещения:** `ClassName(ClassName&& other) noexcept`.
    - Забирает указатели у `other`.
    - **Важно:** Обязательно зануляет указатели в `other`, чтобы его деструктор не удалил память, которой теперь владеем мы.
2. **`std::move`:** Функция, которая ничего не перемещает, а просто делает `static_cast` к rvalue-ссылке, позволяя вызвать move-конструктор для обычного объекта.

Пример:

```
class Buffer {
    int* data;
public:
    // Move Constructor
    Buffer(Buffer&& other) noexcept : data(other.data) {
        other.data = nullptr; // "Обезоруживаем" источник
    }
};
```

Билет 10. Наследование. Спецификаторы доступа. Виртуальное наследование.

1. Наследование

Механизм создания нового класса (`Derived`) на основе существующего (`Base`). Реализует отношение "Is-a" (Является).

2. Спецификаторы доступа при наследовании

Работают как фильтр, понижающий права доступа к членам родителя.

- **`public` наследование:** `public` остается `public`, `protected` остается `protected`. (Самый частый вариант).
- **`protected` наследование:** Всё публичное становится `protected`.
- **`private` наследование:** Всё становится `private`. (Используется для "реализации посредством", аналог композиции).

3. Проблема Ромба (Diamond Problem)

Возникает при множественном наследовании, когда класс наследуется от двух классов, имеющих **общего предка**.

- **Результат:** В конечном классе оказываются две копии полей предка. Неоднозначность и трата памяти.

4. Виртуальное наследование

Решение проблемы ромба.

- **Синтаксис:** `class Derived : virtual public Base`.
- **Суть:** Гарантирует, что в иерархии будет только **один** экземпляр базового класса `Base`.

Билет 11. Наследование: Конструкторы, Деструкторы, Статика.

1. Жизненный цикл (Порядок вызова)

C++ строит объекты "снизу вверх" (от фундамента), а разрушает "сверху вниз".

Конструирование (`Derived d`):

1. Конструктор **Base** (Родителя).
2. Конструкторы **Полей** (Членов класса).
3. Конструктор **Derived** (Тело конструктора потомка).

Разрушение (`~Derived d`):

1. Деструктор **Derived**.
2. Деструкторы **Полей**.
3. Деструктор **Base**.

2. Передача параметров

Конструкторы не наследуются (до C++11). Потомок обязан вызвать конструктор родителя в списке инициализации:

```
Derived(int x) : Base(x) { ... }
```

3. Статические члены

Они **не дублируются** при наследовании. Переменная `Base::staticVar` — это одна и та же ячейка памяти, доступная и `Base`, и `Derived`.

Билет 12. Преобразования вверх и вниз по иерархии.

1. Upcasting (Вверх)

Приведение `Derived*` к `Base*`.

- **Безопасно:** Всегда разрешено неявно. Собака — это Животное.
- **Object Slicing (Срезка):** Если делать Upcast **по значению** (`Base b = derivedObj;`), отрезаются все поля наследника и полиморфизм перестает работать. Используйте указатели или ссылки!

2. Downcasting (Вниз)

Приведение `Base*` к `Derived*`.

- **Опасно:** Не каждое Животное — Собака.

3. Способы Downcasting

1. **`static_cast`**:
   - Compile-time.
   - Быстро, но **не проверяет** реальный тип.
   - Если ошибиться (превратить Кота в Собаку), будет **Undefined Behavior**.
2. **`dynamic_cast`**:
   - Run-time.
   - Использует RTTI для проверки типа.

- Если преобразование невозможно: возвращает `nullptr` (для указателей) или кидает `std::bad_cast` (для ссылок).
- Работает только для полиморфных классов (с `virtual` методами).

Пример:

```
Base* ptr = new Base();
Derived* d = dynamic_cast<Derived*>(ptr); // Вернет nullptr
if (d) { d->onlyDerivedMethod(); }
```

Конец Части 3. *Напиши "дальше", чтобы я отправил Часть 4 (Билеты 13-17).*

Вот **Часть 4** (Билеты 13–17). Здесь мы углубляемся в виртуальные функции, абстрактные классы и отношения между объектами.

Часть 4 из 5: Виртуальные функции и Архитектура (Билеты 13-17)

Билет 13. Виртуальные функции. `virtual`, `override`, `final`.

1. Виртуальная функция

Это метод, поведение которого может быть переопределено в наследниках. Ключевое отличие: при вызове виртуального метода через указатель на базовый класс программа вызывает метод **реального типа объекта**, а не типа указателя.

2. Ключевые слова

- **`virtual`**: Ставится в базовом классе. Включает динамическое связывание.
- **`override`** (C++11): Ставится в наследнике. Просит компилятор проверить, что мы действительно переопределяем существующий виртуальный метод (защита от опечаток в сигнатуре).
- **`final`** (C++11): Запрещает дальнейшее переопределение метода или наследование от класса.

3. Сравнение

Обычная функция	Виртуальная функция
Связывание: Статическое (Early Binding)	Связывание: Динамическое (Late Binding)
Смотрит на: Тип указателя	Смотрит на: Реальный тип объекта
Быстрее (прямой вызов)	Медленнее (через таблицу vtable)

Пример:

```
struct Base { virtual void f() { cout << "Base"; } };
struct Derived : Base { void f() override { cout << "Derived"; } };

Base* p = new Derived();
p->f(); // Выведет "Derived"
```

Билет 14. Таблица виртуальных функций (vtable).

1. Механика работы

Как программа узнает, какую функцию вызвать в рантайме?

1. **vtable (Virtual Table):** Для каждого полиморфного класса компилятор создает статическую таблицу, где хранятся адреса виртуальных функций этого класса.
2. **vptr (Virtual Pointer):** В каждый **объект** компилятор скрытно добавляет указатель на vtable своего класса.

2. Процесс вызова

Когда мы пишем `ptr->func()`:

1. Программа идет по адресу `ptr` в объект.
2. Считывает скрытый `vptr`.
3. Переходит по нему в таблицу `vtable`.
4. Берет адрес нужной функции по индексу.
5. Делает `call` по этому адресу.

3. Роль в `dynamic_cast`

`dynamic_cast` использует информацию о типах (RTTI), которая обычно хранится рядом с `vtable` (по отрицательному смещению), чтобы проверить иерархию наследования.

Билет 15. Чистые виртуальные функции. Абстрактные классы. Интерфейсы.

1. Чистая виртуальная функция

Метод без реализации (или с реализацией, которую нельзя вызвать неявно).

- **Синтаксис:** `virtual void func() = 0;`
- **Смысл:** "Наследник ОБЯЗАН это реализовать".

2. Абстрактный класс

Класс, содержащий хотя бы одну чистую виртуальную функцию.

- **Свойства:** Нельзя создать экземпляр (`Abstract a;` — ошибка). Можно использовать только указатели и ссылки.

3. Интерфейс (в C++)

Чисто абстрактный класс, который содержит:

1. Только чистые виртуальные методы.
2. Виртуальный деструктор.
3. Никаких полей данных.

- Используется для определения контракта взаимодействия ("Что делать", а не "Как").

Билет 16. Виртуальный деструктор.

1. Проблема

При удалении объекта наследника через указатель на базу (`delete basePtr;`):

- Если деструктор **не виртуальный**: Вызывается только `~Base()`. Память наследника течет.
- Если деструктор **виртуальный**: Вызывается `~Derived()`, а затем автоматически `~Base()`.

2. Правило

> Если в классе есть хоть одна виртуальная функция (или класс предназначен для наследования), деструктор **обязан** быть `virtual`.

3. Чистый виртуальный деструктор

`virtual ~Base() = 0;`. Используется, чтобы сделать класс абстрактным, если других методов нет.

- **Важнейший нюанс:** Для него **обязательно нужно написать тело** (реализацию) за пределами класса! Иначе будет ошибка линковки (так как наследники обязаны его вызывать).

Билет 17. Вложенные классы. Агрегирование и Композиция.

1. Вложенные классы (Nested Classes)

Класс, объявленный внутри другого.

- Скрывает реализацию (например, `Node` внутри `List`).
- Имеет доступ к приватным статическим членам внешнего, но не к обычным полям (нужен указатель на внешний объект).

2. Отношения "Has-a" (Содержит)

- **Композиция (Сильная связь):** Часть не живет без целого.
  - Пример: `Автомобиль` и `Двигатель`. Уничтожили машину -> уничтожили двигатель.
  - Код: `Engine eng;` (поле-объект) или `unique_ptr`.
- **Агрегация (Слабая связь):** Часть может существовать отдельно.
  - Пример: `Автомобиль` и `Водитель`. Уничтожили машину -> водитель вышел и пошел домой.
  - Код: `Driver* drv;` (указатель) или `weak_ptr`.

Конец Части 4. *Напиши "дальше", чтобы я отправил финальную Часть 5 (Билеты 18-21).*

Вот **Часть 5** (Билеты 18–21). Финальный блок: множественное наследование, обработка ошибок и приведение типов.

Часть 5 из 5: Множественное наследование и Типизация (Билеты 18-21)

Билет 18. Множественное наследование. Проблема ромба.

1. Множественное наследование

Возможность класса иметь более одного родителя: `class Child : public Mom, public Dad`.

- Объект `Child` содержит поля `Mom` и поля `Dad`.

2. Проблема Ромба (Diamond Problem)

Сценарий: `B` и `C` наследуются от `A`. Класс `D` наследуется от `B` и `C`.

- **Проблема:** В `D` оказываются **две независимые копии** класса `A`.
- **Последствия:** Трата памяти и неоднозначность (какую из двух `A::x` мы меняем?).

3. Решение: Виртуальное наследование

Используем `virtual` при наследовании промежуточных классов:

```
class B : virtual public A { ... };
class C : virtual public A { ... };
class D : public B, public C { ... };
```

- **Результат:** В `D` будет только **один** экземпляр `A`.
- **Важный нюанс:** Конструктор общего предка `A` вызывается **напрямую** из самого последнего класса `D`. Вызовы `A()` из `B` и `C` игнорируются.

Билет 19. Конструкторы и деструкторы при множественном наследовании.

1. Алгоритм вызова конструкторов

Когда создается объект класса-наследника, конструкторы вызываются в строгом порядке:

1. **Виртуальные базовые классы** (если есть). Инициализируются самым производным классом.
2. **Невиртуальные базовые классы** (в порядке перечисления в `class Child : public A, public B`).
3. **Поля класса** (в порядке объявления в классе).
4. **Тело конструктора** самого класса.

2. Алгоритм вызова деструкторов

Строго **обратный** порядок:

1. Тело деструктора.
2. Поля.
3. Невиртуальные базы.
4. Виртуальные базы.

Билет 20. Обработка ошибок (try-catch). Объекты-исключения.

1. Синтаксис

- **`throw obj;`**: Прерывает выполнение, начинает раскрутку стека.
- **`try { ... }`**: Блок, в котором мы ожидаем исключение.
- **`catch (Type& e) { ... }`**: Перехватывает исключение типа `Type`.

2. Правила хорошего тона

1. **Что бросать:** Объекты классов, наследованных от `std::exception` (для полиморфизма и стандартизации).
2. **Как ловить:** Строго **по ссылке** (`const std::exception& e`).
   - *Почему:* Чтобы избежать "срезки" (когда от объекта-наследника копируется только базовая часть `exception`, и теряется информация об ошибке).
3. **`catch (...)`**: Ловит абсолютно все исключения (даже `throw 5;`).

3. Раскрутка стека (Stack Unwinding)

Процесс очистки памяти. При возникновении исключения вызываются деструкторы всех уже созданных локальных объектов в текущей области видимости, затем управление передается выше.

Билет 21. RTTI и Операторы приведения типов.

1. RTTI (Run-Time Type Information)

Механизм идентификации типов в рантайме.

- **`typeid(expr)`**: Возвращает ссылку на `std::type_info`.
- Позволяет узнать имя типа (`name()`) и сравнить типы (`==`).
- Работает корректно только для полиморфных классов (с `virtual`).

2. Операторы приведения (C++ Style Casts)

Вместо небезопасного C-style приведения `(int)x` используются 4 оператора:

1. **`static_cast<T>(expr)`**
   - *Когда:* Compile-time.
   - *Для чего:* Естественные преобразования (`int` <-> `float`), иерархия (`Derived*` -> `Base*`), `void*`.
   - *Безопасность:* Средняя. Не проверяет типы в рантайме при Downcast.
2. **`dynamic_cast<T>(expr)`**
   - *Когда:* Run-time.
   - *Для чего:* Безопасное приведение в иерархии (`Base*` -> `Derived*`).
   - *Результат:* Если тип неверен, возвращает `nullptr` (для указателей) или кидает `std::bad_cast` (для ссылок).
3. **`const_cast<T>(expr)`**
   - *Для чего:* Единственный способ снять или добавить `const`.
   - *Опасно:* Изменение объекта, который изначально был `const`, ведет к UB.
4. **`reinterpret_cast<T>(expr)`**
   - *Для чего:* Битовое переосмысление. Превращает `int*` в `int`, один указатель в другой неродственный.
   - *Опасно:* Максимально. Зависит от железа.

Пример:

```
Base* b = new Derived();
// Безопасно проверяем тип
if (Derived* d = dynamic_cast<Derived*>(b)) {
    d->specificMethod();
```

```
} else {
    // b указывал не на Derived
}
```

Поздравляю! Мы разобрали все 21 билет. У тебя теперь есть полная база для сдачи экзамена. Удачи! 🚀 🎓